

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192312443
Name	P .Ramesh

COURSE OUTCOME COVERED

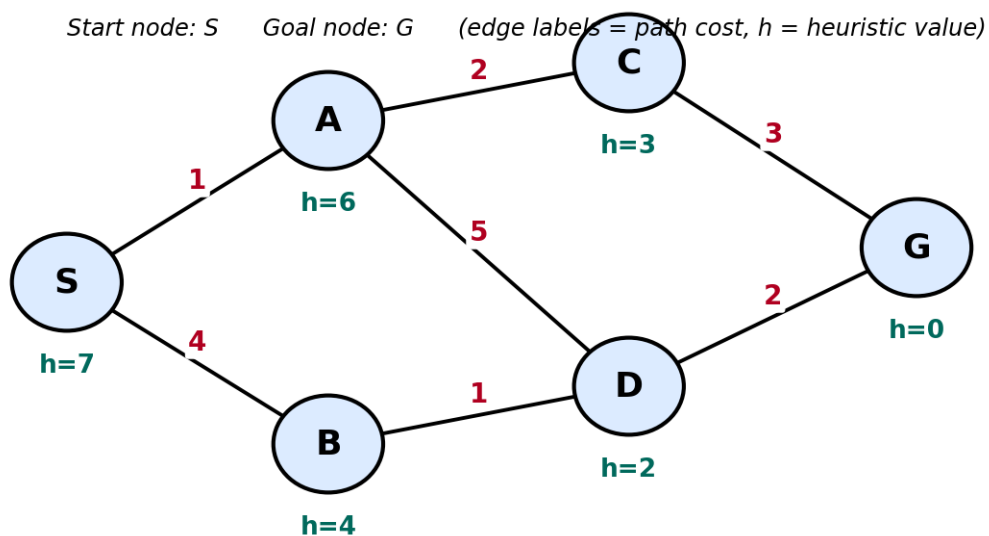
CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

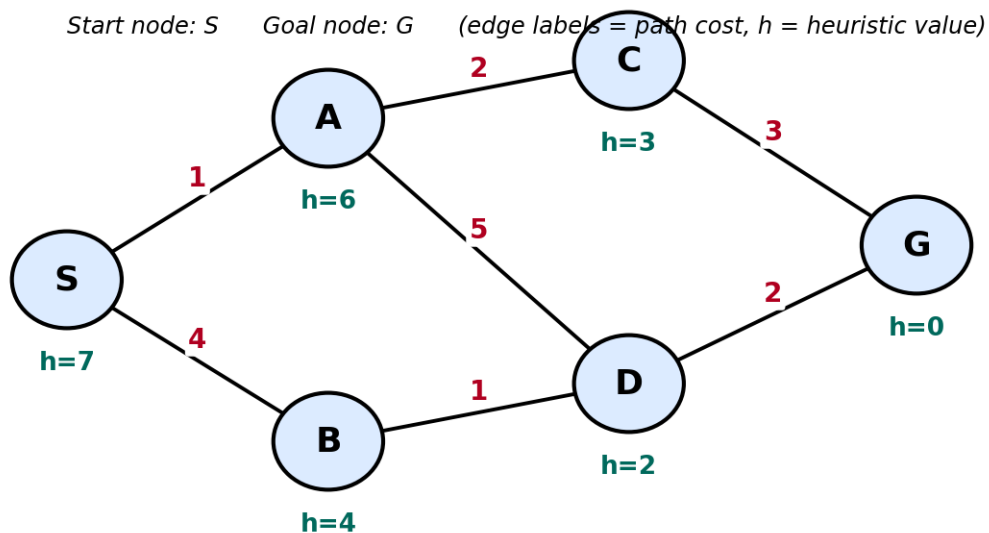
Total Marks:50

Q1. Consider a suitable state-space graph (with heuristic values $h(n)$ assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



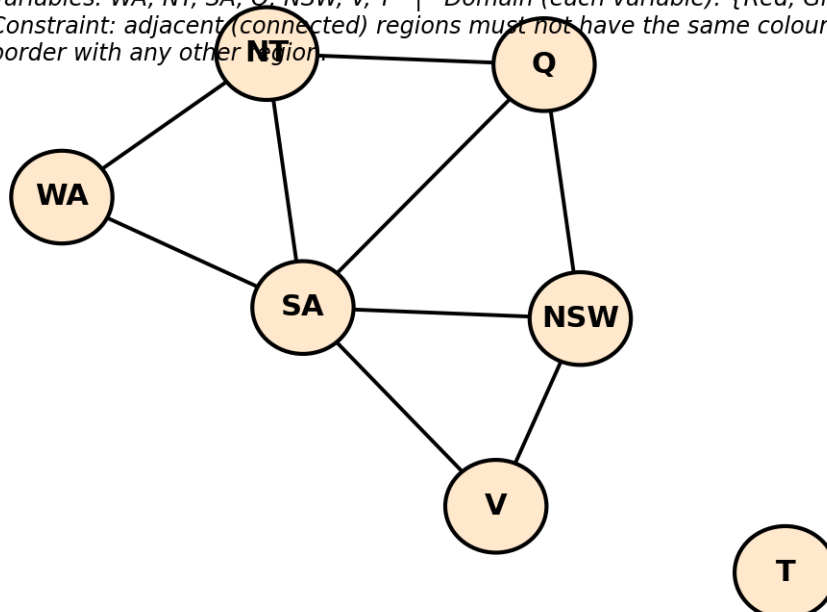
Q2. For a suitable weighted state-space graph (with edge costs and heuristic values $h(n)$) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the $g(n)$, $h(n)$ and $f(n)$ values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



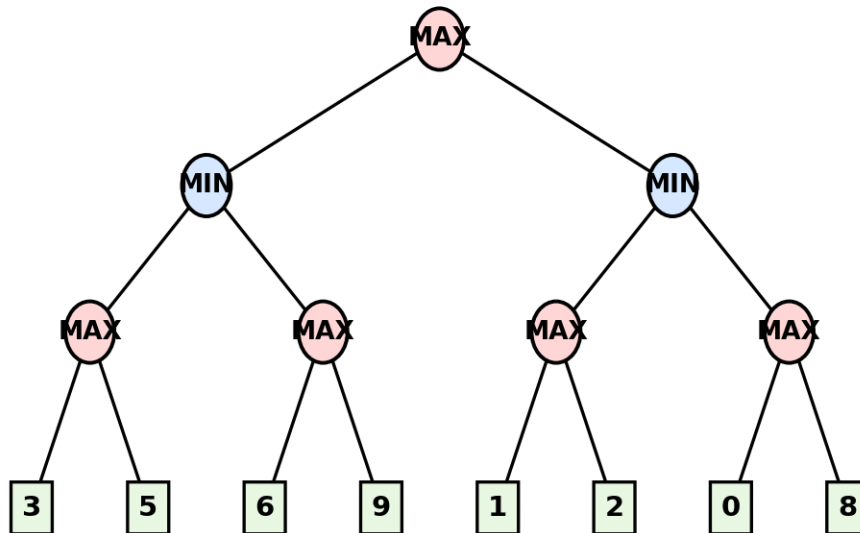
Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}
 Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



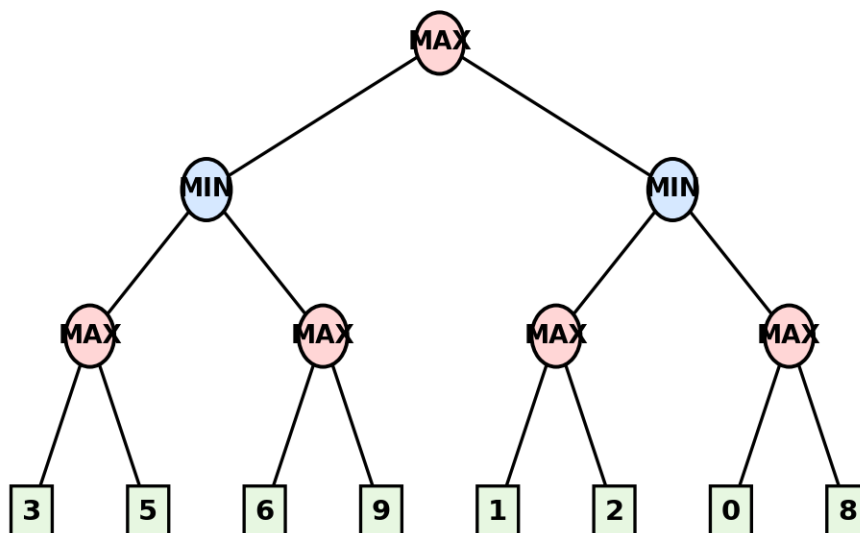
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I P. Ramesh (192312443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P .Ramesh

Dry Run Evaluation**RUBRICS FOR CALCULATION**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	10
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	10
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α - β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the final outcome.	Multiple calculation errors are present that affect the correctness of intermediate steps.	Calculations are mostly incorrect or missing.	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	10
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	10
				Total	50

1)ANS;

Greedy Best-First Search selects the node with the lowest heuristic value $h(n)$ from the frontier (open list).

Heuristic Values

Node $h(n)$

S 7
A 6
B 4
C 3
D 2
G 0

Step 1

Open List: S(7)

Select S for expansion.

Successors:

- A ($h = 6$)
- B ($h = 4$)

Open List: B(4), A(6)

Step 2

Select B (lowest $h = 4$).

Successor:

- D ($h = 2$)

Open List: D(2), A(6)

Step 3

Select D (lowest $h = 2$).

Successor:

- G ($h = 0$)

Open List: G(0), A(6)

Step 4

Select G (Goal Node).

Goal found. Stop.

Frontier/Open List Table

Step Open List Expanded Node

- | | | |
|---|------------|----------|
| 1 | S(7) | S |
| 2 | B(4), A(6) | B |
| 3 | D(2), A(6) | D |
| 4 | G(0), A(6) | G (Goal) |

Final Path

$S \rightarrow B \rightarrow D \rightarrow G$

Path Cost

- $S \rightarrow B = 4$
- $B \rightarrow D = 1$
- $D \rightarrow G = 2$

Total Cost = $4 + 1 + 2 = 7$

2)ANS;

A* Search Dry Run

Formula

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = Cost from Start node to current node
- $h(n)$ = Heuristic value
- $f(n)$ = Evaluation function

Step 1

Start Node S

Node	g(n)	h(n)	f(n)
S	0	7	7

Expand S

Successors:

Node	g(n)	h(n)	f(n)
A	1	6	7
B	4	4	8

Open List: A(7), B(8)

Step 2

Expand A (Lowest f)

Successors:

Node	g(n)	h(n)	f(n)
C	3	3	6
D	6	2	8

Open List: C(6), B(8), D(8)

Step 3

Expand C

Successor:

Node	g(n)	h(n)	f(n)
G	6	0	6

Open List: G(6), B(8), D(8)

Step 4

Expand G

Goal node reached.

Expansion Order

$S \rightarrow A \rightarrow C \rightarrow G$

g(n), h(n), f(n) Summary

Node g(n) h(n) f(n)

S	0	7	7
A	1	6	7
B	4	4	8
C	3	3	6
D	6	2	8
G	6	0	6

Open List at Each Step

Step Open List Expanded Node

1	S(7)	S
2	A(7), B(8)	A
3	C(6), B(8), D(8)	C
4	G(6), B(8), D(8)	G

Final Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Total Path Cost

- $S \rightarrow A = 1$
- $A \rightarrow C = 2$
- $C \rightarrow G = 3$
-

Total Cost = $1 + 2 + 3 = 6$

3)ANS;

1. Variables

The variables are the Australian states and territories:

$V = \{WA, NT, SA, Q, NSW, V, T\}$

2. Domain

Each variable can take one of three colours:

$D = \{\text{Red, Green, Blue}\}$

3. Constraints

Adjacent regions must have different colours.

- $WA \neq NT$
- $WA \neq SA$
- $NT \neq SA$
- $NT \neq Q$
- $SA \neq Q$
- $SA \neq NSW$
- $SA \neq V$

- $Q \neq \text{NSW}$
- $\text{NSW} \neq \text{V}$
- T has no neighbouring region (no constraint)

Dry Run of Backtracking Search

Step 1

Assign **WA = Red**

Constraint Check: Valid

Assignment: {WA = Red}

Step 2

Assign **NT = Green**

Constraint Check:

- $\text{WA} \neq \text{NT}$
- **Assignment:** {WA = Red, NT = Green}

Step 3

Assign **SA = Red**

Constraint Check:

- $\text{WA} = \text{Red}$ Conflict

Backtrack

Try **SA = Green**

Constraint Check:

- $\text{NT} = \text{Green}$ Conflict

Backtrack

Try **SA = Blue**

Constraint Check:

- $\text{WA} \neq \text{SA}$
- $\text{NT} \neq \text{SA}$

Assignment: {WA = Red, NT = Green, SA = Blue}

Step 4

Assign **Q = Red**

Constraint Check:

- $\text{NT} \neq \text{Q}$
- $\text{SA} \neq \text{Q}$
- **Assignment:** {WA = Red, NT = Green, SA = Blue, Q = Red}

Step 5

Assign **NSW = Red**

Constraint Check:

- $\text{Q} = \text{Red}$ Conflict

Backtrack

Try **NSW = Green**

Constraint Check:

- $\text{Q} \neq \text{NSW}$
- $\text{SA} \neq \text{NSW}$

Assignment: {WA = Red, NT = Green, SA = Blue, Q = Red, NSW = Green}

Step 6

Assign **V = Red**

Constraint Check:

- $SA \neq V$
- $NSW \neq V$
- **Assignment:** {WA = Red, NT = Green, SA = Blue, Q = Red, NSW = Green, V = Red}

Step 7

Assign **T = Red**

Constraint Check:

- No neighbouring states
- **Assignment:** {WA = Red, NT = Green, SA = Blue, Q = Red, NSW = Green, V = Red, T = Red}

Constraint Checks and Backtracking

Step	Variable	Assigned Colour	Result
1	WA	Red	✓
2	NT	Green	✓
3	SA	Red	✗ Backtrack
4	SA	Green	✗ Backtrack
5	SA	Blue	✓
6	Q	Red	✓
7	NSW	Red	✗ Backtrack
8	NSW	Green	✓
9	V	Red	✓
10	T	Red	✓

4)ANS;

Minimax Algorithm Dry Run

Step 1: Utility Values at Leaf Nodes

The terminal (leaf) nodes have the following utility values:

Left Subtree:

3, 5, 6, 9

Right Subtree:

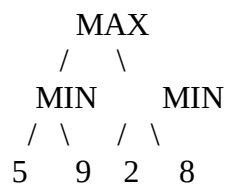
1, 2, 0, 8

Step 2: Evaluate MAX Nodes (Bottom Level)

MAX chooses the maximum value among its children.

MAX Node	Children	Backed-up Value
MAX1	3, 5	5
MAX2	6, 9	9
MAX3	1, 2	2
MAX4	0, 8	8

Tree after MAX evaluation:

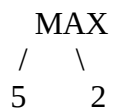


Step 3: Evaluate MIN Nodes

MIN chooses the minimum value among its children.

MIN Node	Children	Backed-up Value
MIN1	5, 9	5
MIN2	2, 8	2

Tree after MIN evaluation:

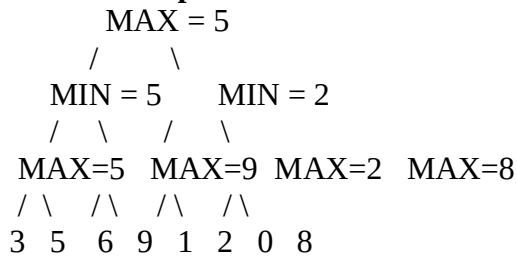


Step 4: Evaluate Root MAX Node

Root MAX chooses the maximum value.

Root MAX	Children	Final Value
MAX	5, 2	5

Final Backed-up Values



Best Move for MAX

The root MAX player selects the left subtree because:

- Left subtree value = 5
- Right subtree value = 2

Since $5 > 2$, MAX chooses the left branch.

Expansion Summary

Level	Player	Selected Value
Leaf	Utility	3, 5, 6, 9, 1, 2, 0, 8
Level 2	MAX	5, 9, 2, 8
Level 1	MIN	5, 2
Root	MAX	5

5)ANS;

Initial Values

- α (Alpha) = $-\infty$
- β (Beta) = $+\infty$

Step 1: Root (MAX)

- $\alpha = -\infty$, $\beta = +\infty$
- Explore Left MIN first.

Step 2: Left MIN

- $\alpha = -\infty$, $\beta = +\infty$

Left MAX (3, 5)

- $\alpha = -\infty$, $\beta = +\infty$
- $\max(3, 5) = 5$

Return 5 to MIN.

MIN updates:

- $\beta = \min(+\infty, 5) = 5$

Right MAX (6, 9)

Current:

- $\alpha = -\infty$
- $\beta = 5$

Evaluate first child = 6

MAX updates:

- $\alpha = \max(-\infty, 6) = 6$

Now,

$$\alpha(6) \geq \beta(5)$$

→ Prune the remaining child (9).

Right MAX returns 6.

Left MIN chooses:

$$\min(5, 6) = 5$$

Return 5 to Root.

Step 3: Root MAX

Current:

- $\alpha = \max(-\infty, 5) = 5$
- $\beta = +\infty$

Explore Right MIN

Step 4: Right MIN

Current:

- $\alpha = 5$
- $\beta = +\infty$

Left MAX (1,2)

Evaluate:

$$\max(1, 2) = 2$$

Return 2

Right MIN updates:

$$\beta = \min(+\infty, 2) = 2$$

Now,

$$\beta(2) \leq \alpha(5)$$

→ Prune the entire Right MAX subtree (0,8).

Right MIN returns 2

Step 5: Root MAX

Compare:

- Left = 5
- Right = 2

MAX chooses 5

Alpha-Beta Values

Node	α	β	Returned Value
Root MAX	5	$+\infty$	5
Left MIN	$-\infty$	5	5
Left MAX	5	$+\infty$	5
Right MAX	6	5	6 (9 Pruned)
Right MIN	5	2	2
Left MAX	5	$+\infty$	2

Node	α	β	Returned Value
Right MAX	—	—	Pruned (0,8)

Pruned Branches

Pruning 1

At Right MAX of Left MIN

```

  MAX
 /  \
6    9

```

After evaluating 6,

$\alpha = 6$ and $\beta = 5$

Since $\alpha \geq \beta$, node 9 is pruned.

Pruning 2

At Right MIN

```

  MIN
 /  \
MAX  MAX
(1,2) (0,8)

```

After Left MAX returns 2,

$\beta = 2$ and $\alpha = 5$

Since $\beta \leq \alpha$, the entire subtree (0,8) is pruned.

Best Move for MAX

- Best Move: Left subtree
- Optimal Utility Value: 5

Justification

Alpha-Beta Pruning eliminates branches that cannot affect the final decision:

- Leaf 9 is pruned because $\alpha = 6 \geq \beta = 5$.
- The entire subtree (0, 8) is pruned because $\beta = 2 \leq \alpha = 5$.

Despite evaluating fewer nodes than standard Minimax, Alpha-Beta Pruning returns the same optimal result.